C Programming - Complete Notes

1. Introduction to C

- C ek **procedural, general-purpose programming language** hai jise Dennis Ritchie ne 1972 mein banaya tha (Bell Labs).
- Ye ek **middle-level language** hai (low-level + high-level features dono).
- C ka use OS (Linux, Windows kernel), embedded systems, compilers banane mein hota hai.

2. Basic Structure of a C Program

```c
#include <stdio.h>   // Header file

int main() {         // Main function - execution yahin se start hoti hai
    printf("Hello, World!\n");  // Output print karta hai
    return 0;         // Program successfully end
}
```

Explanation:

- `#include <stdio.h>` → Standard Input Output library include karta hai.
- `int main()` → Program ka entry point.
- `printf()` → Screen par output print karta hai.
- `return 0;` → Program successfully terminate hua, ye batata hai.

3. Comments

```c
// Single line comment

/* Multi-line
   comment */
```

4. Data Types

Data Type	Size (bytes)	Example
`int`	4	10, -5
`float`	4	3.14
`double`	8	3.14159
`char`	1	'A'
`void`	-	no value

5. Variables

```c
int age = 20;
float price = 99.5;
char grade = 'A';
```

- Variable naming rules: letters, digits, underscore se shuru; number se shuru nahi ho sakta; keywords use nahi kar sakte.

6. Input / Output

```c
int num;
printf("Enter a number: ");
scanf("%d", &num);      // & = address of operator
printf("You entered: %d", num);
```

Format Specifiers

Type	Specifier
int	%d
float	%f
double	%lf
char	%c
string	%s

7. Operators

- **Arithmetic:** `+ - * / %`
- **Relational:** `== != > < >= <=`
- **Logical:** `&& || !`
- **Assignment:** `= += -= *= /=`
- **Increment/Decrement:** `++ --`

8. Conditional Statements

```c
if (age >= 18) {
    printf("Adult");
} else if (age > 0) {
    printf("Minor");
} else {
    printf("Invalid age");
}
```

Switch Case

```c
```

```c
switch (choice) {
    case 1:
        printf("One");
        break;
    case 2:
        printf("Two");
        break;
    default:
        printf("Invalid");
}
```

9. Loops

For Loop

```c
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

While Loop

```c
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

Do-While Loop

```c
int i = 1;
do {
    printf("%d ", i);
    i++;
} while (i <= 5);
```

10. Functions

```c
#include <stdio.h>

int add(int a, int b) {   // Function definition
    return a + b;
}

int main() {
    int result = add(5, 3);   // Function call
    printf("Sum = %d", result);
    return 0;
}
```

- **Function declaration (prototype):** `int add(int, int);`
- Functions code ko reusable aur organized banate hain.

11. Arrays

```c
int numbers[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; i++) {
    printf("%d ", numbers[i]);
}
```

- Array index **0 se start** hota hai.
- 2D Array: `int matrix[3][3];`

12. Strings

```c
```

```c
#include <string.h>

char name[20] = "Rahul";
printf("Name: %s", name);

// Common string functions
strlen(name);      // length
strcpy(dest, src);  // copy
strcat(str1, str2); // concatenate
strcmp(str1, str2); // compare
```

13. Pointers

```c
int x = 10;
int *ptr = &x;   // ptr address of x store karta hai

printf("Value: %d\n", *ptr);   // dereference - value print karega
printf("Address: %p\n", ptr);  // address print karega
```

- `&` → address-of operator
- `*` → dereference operator (value at address)

14. Structures

```c
struct Student {
    char name[50];
    int age;
    float marks;
};

int main() {
    struct Student s1 = {"Amit", 20, 85.5};
    printf("%s %d %.2f", s1.name, s1.age, s1.marks);
    return 0;
}
```

15. File Handling

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("data.txt", "w");  // write mode
    fprintf(fp, "Hello File!");
    fclose(fp);

    fp = fopen("data.txt", "r");  // read mode
    char buffer[100];
    fgets(buffer, 100, fp);
    printf("%s", buffer);
    fclose(fp);

    return 0;
}
```

- Modes: `"r"` read, `"w"` write, `"a"` append, `"r+"` read/write

16. Dynamic Memory Allocation

```c
#include <stdlib.h>

int *ptr = (int*) malloc(5 * sizeof(int));  // memory allocate
free(ptr);  // memory free karna zaroori hai
```

- `malloc()`, `calloc()`, `realloc()`, `free()`

17. Important Keywords

`int, float, char, double, void, if, else, switch, case, for, while, do, break, continue, return, struct, typedef, const, static, sizeof`

18. Common Mistakes to Avoid

- Semicolon `;` bhoolna
- `=` aur `==` mein confuse hona
- Array index out of bounds

- Uninitialized pointers use karna
- Memory allocate karke `free()` na karna

Quick Tip for VS Code

1. C Extension install karo: **"C/C++"** by Microsoft
2. Code compiler ke liye **MinGW (Windows)** ya **GCC (Linux/Mac)** install karo
3. Terminal mein run karne ke liye:

```
gcc filename.c -o output
./output
```